Chapter 2
A Gentle Introduction to Event-B

This chapter addresses Event-B from a practical modelling perspective. For a complete theoretical account of Event-B, the reader is encouraged to consult [1]. We present a series of modelling examples using Event-B. The starting point of each example is a small problem to solve. Each example discusses and demonstrates how to use Event-B to model a particular system or small program. Event-B language is based on predicate calculus, so the language includes many interesting set and relation operators. Not only do we discuss the operators and their semantics but also demostrate their usability from a modelling viewpoint. Code excerpts cover topics such as social networks, dictionaries, and sorting.

The syntax underlying the discussed example forms the backbone of the more complex examples discussed in later chapters.

2.1 Event-B Modelling Process

Event-B is a formal modelling language for reactive systems, introduced by Jean-Raymond Abrial [1], which allows the modelling of complete systems (software and hardware systems). Event-B is based on *Action Systems* [3], a formalism describing the behaviour of a system by the (atomic) actions that the system carries out. An Action System describes the state-space of a system and the possible actions that can be executed in it. Event-B is intended for the rigorous development of *Dependable Systems* by enabling precise modelling and mathematically-proven correctness. The Event-B language allows us to describe a single framework composed of a system and the way it reacts to its environment. The state-space of an Event-B system is defined by a collection of variables. Atomic actions, called *events*, are *triggered* as the system reacts to its environment. The effect of each action is a transition in the state-space. An Event-B model comprises a static part, called *context*, and a dynamic part called *machine*. The model of a system can include several machines and contexts. The static part of a model defines the context of the system (constants and uninterpreted sets and their properties expressed as **axioms**) and the state-space (variables, their types and

properties). The dynamic part defines all possible events, including the **initialisation** event computing the initial state.

An event is composed of a *guard* and an *action*. The guard (written between keywords **where** and **then**) represents conditions that must hold in a state for the event to be enabled. The action (written between keywords **then** and **end**) computes new values for state variables, thus performing an observable state transition. If the system reaches a state where no event guard holds, it halts and is said to have *deadlocked*. There is no requirement that the system should halt, and indeed, most Event-B models represent systems that run forever. If halting is desired, the system can be modelled using **convergent** events that strictly decrease the value of a natural number expression called the machine **variant**. **convergent** events ensure that the system cannot make infinite progress without advancing towards completion. They can only be triggered in states where the value of the **variant** is non-negative. Additionally, the system may reach a state in which the guards of more than one event hold. In this situation, the system is said to be *non-deterministic*: Event-B semantics allows any of the events whose guards are satisfied to be triggered.

In Event-B, systems are typically modelled via a sequence of refinements. First, an abstract machine is developed and verified to satisfy whatever correctness and safety properties are desired. Refinement machines are used to add more detail to the abstract machine until the model is sufficiently concrete for hand or automated translation to code. Refinement in Event-B preserves safety properties: every refinement machine must satisfy the invariant properties established at the abstract level. Refinement proof obligations are discharged to ensure that each refinement is a faithful model of the previous machine, so that all machines satisfy the correctness properties of the original model. Event-B models at different abstraction levels serve different purposes: at higher levels, models are used to state and verify key system properties; at lower levels, models guide or underpin the system behaviour. A model M_{i+1} at stage $i+1$ is said to be a refinement of a model M_i when the states computed by M_i and M_{i+1} at each stage obey a so-called *gluing invariant* [1]. A gluing invariant relates variables of the refinement model M_{i+1} to variables of the refined model M_i, stating properties of the joint behaviour of both models. Therefore, the abstract (refined) model can be replaced with the refinement model without any observable difference to users of the abstract model. Gluing invariants *i*) are invariants defined in the refinement model, *ii*) they relate the state of the refinement model with the state of the refined model, *iii*) they must be preserved by the refinement model, therefore, the declaration of a gluing invariant generates proof obligations, which are sufficient conditions that guarantee that both, although at different levels of abstraction, are models of the same system[1].

2.2 Event-B Mathematical Notation

A selection of Event-B set and relation notation is shown in Table 2.1. We use square brackets to apply (evaluate) a relation to a set of elements. For instance, r[s] applies the

[1] Example 2.16 in Page 13, and Example 2.17 in Page 14 discuss gluing invariants (see Section 2.5).

Exp.	Name	Definition	Short Form
q; r	forward composition	$\{(x,z) \mid \exists y \cdot (x,y) \in q \ \wedge \ (y,z) \in r\}$	q; r
id[s]	identity relation	$\{(x,y) \mid (x,y) \in s \times s \ \wedge \ x = y\}$	**id**[s]
s ⊲ r	domain restriction	$\{(x,y) \mid (x,y) \in r \wedge x \in s\}$	**id**[s]; r
s ⩤ r	domain subtraction	$\{(x,y) \mid (x,y) \in r \wedge x \notin s\}$	(**dom**(r) \ s) ⊲ r
r ⊳ s	range restriction	$\{(x,y) \mid (x,y) \in r \wedge y \in s\}$	**id**[s]; r
r ⩥ s	range subtraction	$\{(x,y) \mid (x,y) \in r \wedge y \notin s\}$	r ⊳ (**ran**(r)\s)
r[s]	image	$\{y \mid (x,y) \in r \wedge x \in s\}$	**ran**(s ⊲ r)
r ⊕ q	relation overriding	$\{(x,y) \mid (x,y) \in q \ \vee \ ((x,y) \in r \wedge \nexists z \cdot (x,z) \in q)\}$	q ∪ (**dom**(q) ⩤ r)
r˜	inverse relation	$\{(x,y) \mid (y,x) \in r\}$	r˜

Table 2.1: Basic Event-B mathematical notation

relation r to all of the elements in set s. The result of r[s] is a subset of the range of the relation r. Event-B provides standard notation for set union, intersection, difference, etc. The symbol × denotes the *cross product* between two sets. The operator **dom** returns the domain of a relation, and **ran** its range. The operator **id**[s] denotes the *identity relation* calculated over the set of elements s. Applying the *forward composition* relation q; r to an element x in the domain of relation q returns a set of elements calculated as the result of applying r to q[{x}]. When q is a function, we use the notation q(a) for applying q to a, therefore, q(a) is the *image* of a through q. The *domain restriction* relation expression s ⊲ r restricts the domain of a relation r to a subset s of its domain. The *range restriction* relation expression r ⊳ s restricts the range of relation r to only elements in a subset s of its range. *Domain subtraction* is defined similarly to domain restriction, except that the elements in the set s are excluded rather than considered. *Range subtraction* is defined similarly to domain subtraction, except that it applies to elements in the range of the relation rather than to elements in its domain. The *overriding* r ⊕ q of a relation r by a relation q returns a relation formed by taking all of the pairs of q and adding all of the pairs of r whose first elements are not in the domain of q, *i.e.*, q ∪ (**dom**(q) ⩤ r). Event-B also provides the r˜ notation for the inverse of a relation r. Symbol \ denotes set difference. Event-B relations are represented as sets of pairs.

2.3 Event-B Relations and Functions

Relations are sets of pairs. A relation f with domain A and range B is denoted f: A ↔ B. If f is a function defined for all of the elements of A, we say that f is a *total function*, and we write f: A → B. If f is defined for some values of A, we say that f is a *partial function*, and we write f: A ⇸ B. If f is a function such that no element in its range is associated with more than one element in the domain of f, then we say that f is an *injective function*, and we write f: A ⤔ B. If f is a function whose range is B, we say that f is a *surjective function*, and we write f: A ⤀ B. If f is both injective and surjective, then f is a *bijection*, and we write f: A ⤖ B.

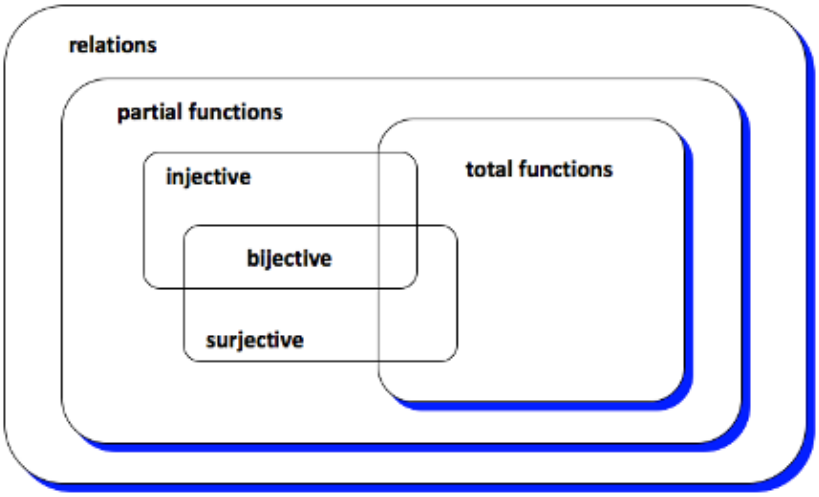

Fig. 2.1: Relations and functions.

Figure 2.1 shows the relative hierarchy of types of relations and functions. Any function is a relation. Functions can be total or partial. Every total function is a partial function. Functions can be injective or surjective. Some surjective functions are partial and other ones are total. Some injective functions are partial and other ones are total. Bijective functions are both surjective and injective functions. Some bijective functions are partial and other ones are total. Although the term bijection is often implicitly understood as a *total* function that is both injective and surjective. In Event-B, however, totality is treated as an independent property. A bijective function in Event-B is defined as a function that is both injective and surjective, but it may be either partial or total. As a consequence, partial bijections are permitted. When totality is required, it must be stated explicitly.

2.4 Event-B Syntax

Example 2.1 (Events)**.** The following construct shows the syntax of events:

$$\textbf{any } x \textbf{ where } G(s,c,v,x) \textbf{ then } v := BA(s,c,v,x) \textbf{ end}$$

It specifies an event that can be triggered in a state where the guard $G(s,c,v,x)$ holds for some bounded set of variables x, given sets s, constants c, and machine variables v. When the event is triggered, a valuation of x satisfying $G(s,c,v,x)$ is non-deterministically chosen, and the event action $v := BA(s,c,v,x)$ is executed with that valuation, where BA is a *before-after* predicate that relates v pre- and post-state values. Event actions

assign into machine variables. The event assignment binds v to values that adhere to BA. Event-B provides some simplified syntaxes for events. For example, if x is empty, the event does not include an **any** clause. The event does not include a **then** clause if BA is missing.

Event actions $v := BA(s,c,v,x)$ compute new post-state values for the machine variables v, thus performing an observable state transition. They constitute the body of an event and are written between the keywords **then** and **end**.

Example 2.2 (Using sets for abstract modelling)**.** We want to model dictionaries in Event-B and are only interested in adding words to this dictionary. For the sake of simplicity, we assume that a word has a sole meaning. In that respect, our dictionary behaves more like a function. We choose that function to be *partial* so that we have a small dictionary that does not include an entry for every word of the language. A word is a set, so is . A dictionary is a partial function from words to meanings.

@inv1 word ⊆ WORD
@inv2 meaning ⊆ MEANING
@inv3 dictionary ∈ word ⇸ meaning

We show below an event that adds a new word and its meaning to the dictionary. Guard @grd1 checks that the word w is in the set difference WORD \ word; since w cannot be in word, it needs to be a new word taken from WORD. Likewise, guard @grd2 checks that m is a new meaning that is not already in the set of meanings. Event action @act1 adds w to the set of words, @act2 adds m to the set of meanings, and @act3 adds to the dictionary an entry that maps word w into meaning m.

event add-word
any w m
where
@grd1 w ∈ WORD \ word
@grd2 m ∈ MEANING \ meaning
then
@act1 word := word ∪ {w}
@act2 meaning := meaning ∪ {m}
@act3 dictionary(w) := m
end